

Ulsan-Fault earthquake catalog pipeline

Reference manual: detection → association → location → relocation (HypoDD dt.ct + dt.cc)

Long-term (2010–present) catalog for the Ulsan Fault, SE Korea

src/ufpipe + detection_test

The full workflow is **two pipelines** that share the same method. **(A)** the `ufpipe` package (`src/ufpipe`) builds the *long-term absolute-location catalog* (detection→association→HYPOINVERSE). **(B)** the `detection_test` pipeline takes it the last step to *double-difference relocation* (HYPOINVERSE→QC→HypoDD dt.ct+dt.cc), and also runs the 4-picker comparison. This manual covers **both**, end to end. The relocation pipeline additionally has a dedicated guide (`detection_test/reloc_2016_uf/PIPELINE.md` + `study_guide.pdf`).

Contents

1	What the pipeline does	1
1.1	Two pipelines, one workflow	2
1.2	Two orthogonal “model” dimensions	2
2	Install & environment	2
3	The catalog stages (pipeline A)	3
3.1	Source of truth for pick↔event assignment	3
4	Parameters (src/ufpipe/config.py — the single disclosed source)	3
4.1	Detection / DSP	4
4.2	Association (PyOcto) — REGION (lenient) vs REGION_STRICT	4
4.3	Velocity model & location	4
5	How to run	5
5.1	Running each stage separately (pipeline A)	5
5.2	Relocation (pipeline B) — running the last stage	6
6	Outputs & layout	6
7	Relocation — HypoDD dt.ct + dt.cc (pipeline B)	7
8	Robustness & invariants	8
9	Troubleshooting / provenance checks	8

1 What the pipeline does

Build an **internally consistent, long-term earthquake catalog** for the Ulsan Fault region (SE Korea), 2010–present, from continuous seismic waveforms, using AI phase pickers so small events are detected uniformly even as the recording network grows, and finally **relocate** the catalog with double-difference (HypoDD dt.ct + dt.cc) for sub-100 m relative precision. Every stage is scripted and parameterized; results reproducible.

Detection	Association	Location	QC	Relocation
PhaseNet / PhaseNet+	PyOcto (+augment)	HYPOINVERSE	uf_cluster	HypoDD dt.ct + dt.cc
<i>picks</i> →	<i>events+assign</i> →	<i>.sum/.arc</i> →	<i>well-located</i> →	<i>hypoDD.reloc</i>

1.1 Two pipelines, one workflow

	(A) ufpipes (src/ufpipe)	(B) detection_test
Product	long-term absolute-location catalog	double-difference relocated catalog (+ picker comparison)
Stages	detection → association → augment → PHS → locate	detection → association → HYPOINVERSE → QC → HypoDD dt.ct+dt.cc
Networks	KS, KG	KS, KG, GJ (Gyeongju temporary array)
Association	whole-year PyOcto	per-month PyOcto (daily-chunked)
Driver	<code>python -m ufpipes.run_pipeline</code>	<code>detection_test/reloc_2016_uf/run_picker_reloc.py</code>
Ends at	HYPOINVERSE (§3–§6)	HypoDD dt.cc (§7)

They share the same *method* (same pickers, PyOcto, HYPOINVERSE, kim velocity models) but are separate code paths with different products. **To reach dt.cc relocation you use pipeline (B)** — see §7.

1.2 Two orthogonal “model” dimensions

Dimension	Flag	Values
Picker model	<code>--model</code>	<code>stead</code> , <code>original</code> , <code>phasenet_plus</code> , <code>eqt</code> , ...
Velocity model	<code>--velmodel</code>	<code>kim1983</code> , <code>kim2011</code>

Any picker model can be located with any velocity model. `stead/original` are SeisBench PhaseNet weights; `phasenet_plus` is EQNet PhaseNet+ (needs a local EQNet clone).

2 Install & environment

One-time setup

- `conda env create -f environment.yml && conda activate ulsan` (the code also runs in base here)
- `pip install torch --index-url https://download.pytorch.org/whl/cu128` (match your CUDA; GPU strongly recommended)
- `pip install -e .` (installs the `uflib` + `ufpipe` packages, editable)
- HYPOINVERSE**: the external `hyp1.40` binary must be on `PATH` (not pip-installable).
- PhaseNet+ only**: a local EQNet clone at `config.EQNET_DIR`.

After `pip install -e .`, `import ufpipes.config` and `python -m ufpipes.run_pipeline` work from any directory — no `sys.path` hacks.

3 The catalog stages (pipeline A)

STAGES = [detection, association, augment, phs, locate] — run in order. Each is a function in `src/ufpipe/core.py`; `run_pipeline.py` chains them per year.

#	Stage	In → Out	What it does
1	detection	continuous mSEED → picks/ (per station-day)	AI picker on each station-day; anti-alias >100 Hz, resample to 100 Hz, pick P/S above threshold. GPU. Idempotent (skips done days).
2	association	picks → pyocto_*.csv + assignment	PyOcto octree associator groups picks into events (gate below). Whole-year or daily-chunked.
3	augment	assignment → augmented assignment	scan daily picks for orphans PyOcto missed, add to each event so HYPOINVERSE sees the full pick set. Backs up + overwrites in place; idempotent.
4	phs	assignment → UF{year}.phs	write the HYPOINVERSE phase file from the (augmented) assignment.
5	locate	.phs → UF{year}.{sum,arc}	run HYPOINVERSE (hyp1.40) with the chosen velocity model → located catalog + errors.

Pipeline order matters

augment runs *between* **association** and **phs**, so HYPOINVERSE sees the full augmented pick set and computes correct GAP/DMIN/ERZ/RMS. Do not reorder.

3.1 Source of truth for pick↔event assignment

The **PyOcto assignment (after augmentation)** is authoritative for which (station, phase) tuples belong to each event. Do NOT use a legacy time-window pick dump — when two events fall <60 s apart at the same station the earliest pick “wins” and leaks into the wrong event. Downstream (SAC-header writers, magnitude/QC notebooks) must read `pyocto_assignment_{velmodel}_{year}.csv`.

4 Parameters (src/ufpipe/config.py — the single disclosed source)

Change defaults here. Everything is one file.

4.1 Detection / DSP

Parameter	Value	Meaning
SAMPLING_RATE	100.0 Hz	the pickers' trained rate; all data resampled to it
ANTIALIAS	lowpass 45 Hz, 8 corners, zerophase	applied BEFORE down-sampling (>100 Hz KG/NS)
BANDPASS	1–40 Hz, 4 corners, causal	legacy detection path; PhaseNet+ runs raw
P_THRESHOLD, S_THRESHOLD	0.2	SeisBench pick probability threshold
PNPLUS_MIN_PROB	0.3	EQNet PhaseNet+ pick threshold
PNPLUS_NT	1024×36	PhaseNet+ patch length (whole-day, cut_patch)
MAX_CORES	24	CPU cap (respects taskset affinity)

4.2 Association (PyOcto) — REGION (lenient) vs REGION_STRICT

Parameter	REGION	REGION_STRICT	Meaning
lat / lon	(34.5,37.0) / (128.5,130.0)		search area (deg)
zlim	(0, 40)		depth range (km)
time_before	300		origin-search window (s)
n_picks	4	6	min total picks
n_p_picks	2	3	min P picks
n_s_picks	2	3	min S picks
n_p_and_s_picks	1	2	min stations with both P+S (depth constraint)
pick_match_tolerance	1.5	1.0	residual cap (s) — the primary quality knob
min_node_size / _location	10/1.5	2.0/0.5	octree granularity (km)
refinement_iterations	3	8	localise+rematch cycles
n_threads	16		PyOcto threads (polite on a shared box)

Pass `--strict` to use `REGION_STRICT` (fewer, cleaner events). Velocity for association = `PYOCOT_VELMODEL` (kim1983).

4.3 Velocity model & location

`DEFAULT_VELMODEL` = kim2011; kim1983 also available (`data/metadata/velocity_model/kim1983.csv`, a layered V_p/V_s model). `PHASE_CHANNELS` = {P: HHZ, S: HHN}.

5 How to run

Standard runs (from the repo root)

<code>python -m ufpipes.run_pipeline --model original --years 2024</code>	one year, all 5 stages
<code>python -m ufpipes.run_pipeline --model original --years 2010-2024</code>	the whole record
<code>python -m ufpipes.run_pipeline --model original --years 2015,2016 --velmodel kim1983</code>	specific years, kim1983
<code>python -m ufpipes.run_pipeline --model original --years 2024 --stage-from association</code>	resume (picks already done)
<code>python -m ufpipes.run_pipeline --model original --years 2024 --days 1-3</code>	detection on a few days (testing)
<code>python -m ufpipes.run_pipeline --model original --years 2024 --strict</code>	strict association gate

CLI flags: `--model` · `--years` ('2010-2024' or '2015,2016') · `--velmodel` · `--days` · `--device` {cuda,cpu} · `--workers` · `--strict` · `--force` (allow writing to `stead`) · `--stage-from` {detection,association,augment,phs,locate} (start mid-chain).

Shared 64-core server — always pin cores

Detection sizes its preprocessing pool and `torch` threads from the process CPU *affinity*, capped by `MAX_CORES=24`. Launch under `taskset` so the whole job stays within budget:

```
OMP_NUM_THREADS=1 taskset -c 0-7 python -m ufpipes.run_pipeline --model original --years 2010-2024
```

Without pinning, PhaseNet+ grabbed ~49 cores / 193 threads and starved everything. Inference is **GPU-preferred** (warns loudly, never silently falls back to CPU).

Detection is **idempotent** — it skips station-days whose picks already exist, so runs resume safely after an interruption.

5.1 Running each stage separately (pipeline A)

Every `ufpipe` stage is also a standalone CLI (`--model` `--year`), so you can run detection alone, then association alone, etc. Order is `detection` → `association` → `augment` → `phs` → `locate`. Each reads the previous stage's output from disk.

Stage	standalone command
Detection	<code>python -m ufpipes.detection --model original --year 2024 [--days 1-5] [--workers 8] [--device cuda]</code>
Association	<code>python -m ufpipes.association --model original --year 2024 [--strict]</code>
Augment	(runs inside <code>run_pipeline</code> ; or call <code>core.run_augment_year</code> from Python)
PHS	<code>python -m ufpipes.make_phs --model original --year 2024</code>
Location	<code>python -m ufpipes.run_hypoinverse --model original --year 2024 --velmodel kim2011</code>

Or drive the same stage functions from Python / a notebook:

```
from ufpipes import core, config
core.run_detection_year("original", 2024, days=range(1,6))
events, assign = core.run_association_year("original", 2024)
core.write_phs("original", 2024); core.run_hypoinverse_year("original", 2024,
velmodel="kim2011")
```

5.2 Relocation (pipeline B) — running the last stage

The relocation pipeline (`detection_test/`) has its own driver and its own detection/association (per-month, GJ-inclusive). It is **year-general** (`--year`, default 2016). Run from `detection_test/reloc_2016_uf/`:

Step	command
Readiness	<code>python preflight_year.py --year 2016</code>
Detection	<code>python ../lib/run_seisbench_picker.py --model original --month 2016-01</code> (loop months; PN+: <code>run_pnplus_month.py</code>)
Association	<code>python ../lib/associate_daily.py --picker original --month 2016-01</code>
Reloc (to QC)	<code>python run_picker_reloc.py --picker original --year 2016 --through hypoinverse</code> (fast; SAC→HYPOINVERSE→QC)
Reloc (full)	<code>python run_picker_reloc.py --picker original --year 2016 --through dtcc --clean-cache</code> (+ rereference→xcorr→HypoDD dt.cc; xcorr ~6 h dense)

`--through hypoinverse` stops at the QC'd absolute-location subset; `--through dtcc` continues to the double-difference relocation. `--clean-cache` deletes the ~tens-of-GB interp cache after dt.cc (essential for multi-year runs). Full detail: `PIPELINE.md` + `study_guide.pdf`.

6 Outputs & layout

All pipeline products go under `outputs/models/<model>/` (regenerable; gitignored):

Path	Contents
.../detection_location/<year>/picks/	per-station-day picks (stage 1)
.../pyocto/pyocto_{vm}_{year}.csv	associated events (stage 2)
.../pyocto/pyocto_assignment_{vm}_{year}.csv	pick→event assignment (authoritative)
.../station_table/stations_{year}.csv	per-year station roster (discovered from metadata)
.../HypoInv/PHS/UF{year}.phs	HYPOINVERSE phase file (stage 4)
.../HypoInv/{vm}/UF{year}.{sum,arc,prt}	located catalog + errors (stage 5)

Inputs (restructured 2026-07): waveforms in `KS_KG/` (station dirs at root, parallel to `GJ/NS/`); station metadata in `data/metadata/ks_kg_station_table/`; velocity model in `data/metadata/velocity_model/`; HYPOINVERSE control inputs (`STA/kim*`) in `data/hypoinv/`.

7 Relocation — HypoDD dt.ct + dt.cc (pipeline B)

The final step: turn the QC'd absolute-location catalog into a **double-difference relocated** catalog for sub-100m relative precision. Driven by `detection_test/reloc_2016_uf/run_picker_reloc.py --through dtcc`. Stages (after detection + association, which pipeline B runs per-month, GJ-inclusive):

#	Stage	What it does
1	SAC store	event-idx-keyed native-rate SAC waveforms from the associated picks (<code>event_sac/</code>)
2	HYPOINVERSE	absolute location of all UF-box events (<code>kim2011</code>) → <code>.sum</code>
3	QC	<code>uf_cluster.QC</code> : keep <code>erh<5</code> , <code>erz<5</code> , <code>gap<270</code> , <code>num>5</code> , <code>rms<1.0</code>
4	re-reference	stamp each SAC's origin from the (full-run) HYPOINVERSE <code>.sum</code>
5	ph2dt	build <code>event.dat</code> + catalog differential times <code>dt.ct</code>
6	xcorr (GPU)	cross-correlation differential times <code>dt.cc</code> (band 5–20 Hz, interp 1000 Hz, <code>cc≥0.7</code>)
7	HypoDD	<code>kim2011</code> , <code>ISTART=2</code> , <code>ISOLV=2</code> (LSQR), adaptive damping (CND 40–80) → <code>hypoDD.reloc</code>

Relocation outputs (symlinked into the working dir)

Each run publishes `detection_test/reloc_<year>_uf[_<p>]/results/` with symlinks to the external run tree: `hypoDD.reloc.dtcc` (`dt.ct+dt.cc`), `hypoDD.reloc.dtct` (`dt.ct-only`), `HypoInv.full.sum`, ...“`dt.cc-resolved`” = events keeping ≥ 1 surviving cross-correlation link — the highest-precision subset.

The invariant that matters (see PIPELINE.md)

Origins flow into `dt.cc`: `rereference` stamps SAC origins from a `.sum`, and `xcorr` stores $dt = (t_1 + \text{shift} - ot_1) - (t_2 - ot_2)$ — subtracting those origins. Use the *full-run* HYPOINVERSE solution QC gated on (never a redundant re-run), or the `dt.cc` *values* are corrupted. This is enforced by `run_picker_reloc.py::inject_full_hypoinverse`.

8 Robustness & invariants

- **Network growth is automatic.** Stations are *discovered from StationXML + on-disk coverage* each month; the deployed network for that period is what enters the run. No hard-coded station lists.
- **Mixed sampling rates handled.** >100 Hz stations (KG/NS at 200 Hz) are anti-alias-lowpassed then resampled to the pickers' 100 Hz. The anti-alias filter is applied BEFORE down-sampling (never after).
- **Velocity model, thresholds, and gates are epoch-invariant by choice** $\Rightarrow$ the catalog stays internally comparable across years.
- **Idempotent & resumable** at every stage; `--stage-from` restarts mid-chain.
- **Do not edit the stead reference run** (it is the baseline). Scripts refuse `--model stead` writes unless `--force`.

9 Troubleshooting / provenance checks

Symptom	Check
Detection finds no waveforms	<code>python -c "import ufpipes.config as c,os; print(c.CONTINUOUS, os.path.isdir(c.CONTINUOUS))"</code> — must point at KS_KG/ and exist.
<code>ImportError: ufpipes / uflib</code>	<code>pip install -e .</code> in the active env; <code>import ufpipes.config; print(__file__)</code> must be <code>src/ufpipes</code> .
Wrong pipeline imported	<code>ufpipes</code> is renamed precisely to avoid the collision with the external korea-cluster-relocation <code>pipeline</code> . Verify <code>ufpipes.config.__file__</code> is under <code>src/</code> .
Events look over/under-associated	compare <code>REGION</code> vs <code>--strict</code> ; the <code>pick_match_tolerance</code> is the primary knob.
GPU not used	inference is GPU-preferred and warns loudly; check the torch/CUDA install (never silently CPU).
Sparse early years, few events	expected — KS/KG-only network (2010–2016) has larger azimuthal gaps; the QC gate honestly reflects what the network resolves.

Companion	docs:	docs/overview.md,	docs/pipeline.md,	docs/how-to-run.md,
docs/directory-structure.md	·	src/ufpipes/README.md	·	relocation pipeline:
detection_test/reloc_2016_uf/PIPELINE.md	+ study_guide.pdf.			